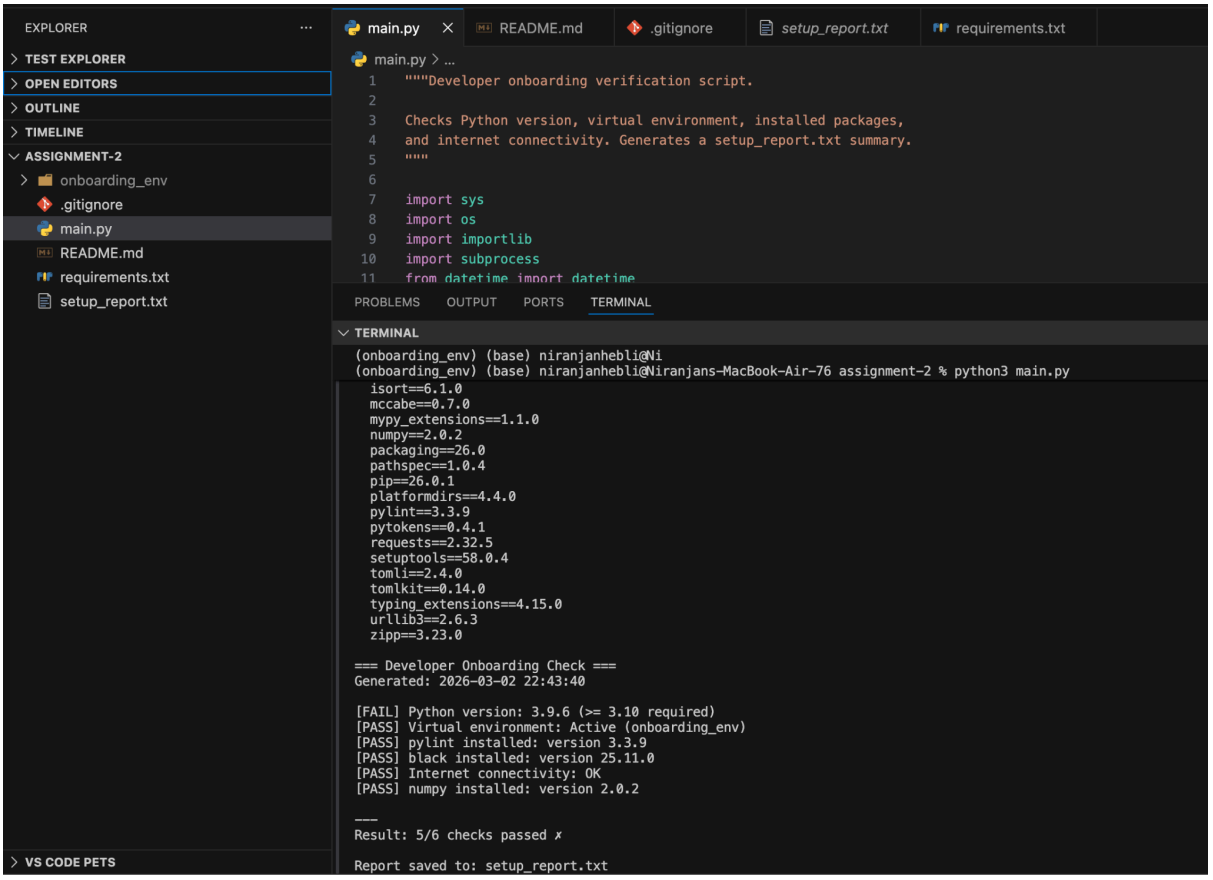
Python Dev Environment Setup

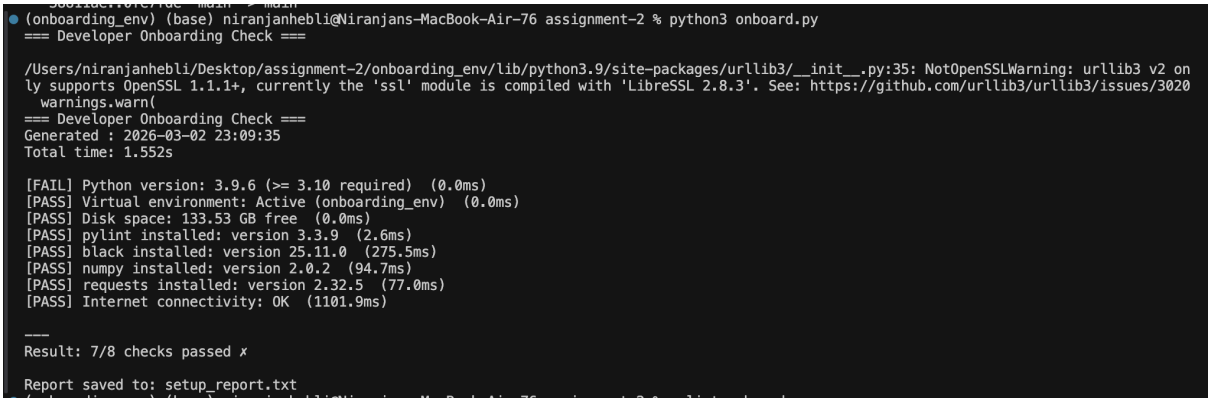
Part A: Concept Application

Github Link:- https://github.com/NiranjanHebli/DAY7_AM_Assignment

Output Screenshot:-



Part B: Stretch Problem



Part C: Interview Ready

Q1: Conceptual — Virtual Environments

A Python virtual environment is like a **sandbox** where developers can install packages without affecting the rest of the system. Developers use them to avoid conflicts between projects, since different projects may need different versions of the same library. Without one, installing packages globally can cause version clashes and break other projects.

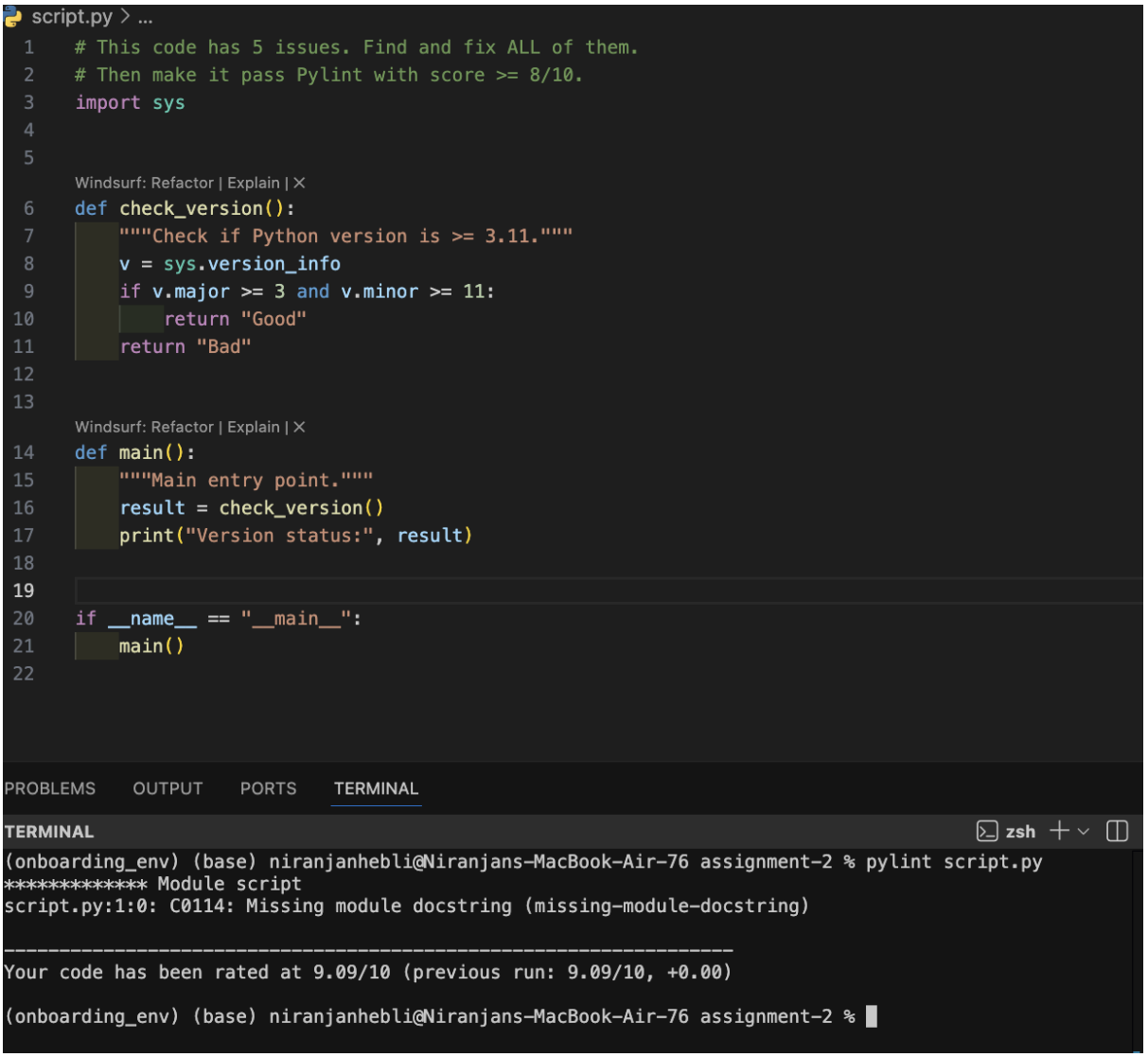
Q2: Coding — Fix and improve this code

```
# This code has 5 issues. Find and fix ALL of them.
# Then make it pass Pylint with score >= 8/10.

import os, sys, json

def checkVersion():
    v = sys.version_info
    if v.major >= 3 and v.minor>=11:
        return "Good"
    else:
        return "Bad"

result = checkVersion()
temp = 42
print( "Version status:" , result )
```

List the 5 issues and provide the corrected code.

The 5 Issues Identified

1. Multiple imports on one line (import os, sys, json) → violates PEP8 style.
2. Unused imports (os, json) → should be removed.
3. Function name not in snake_case (checkVersion) → should be check_version.
4. Unused variable (temp = 42) → unnecessary, lowers lint score.
5. Docstrings missing → Pylint will complain about missing function/module docstrings.



Q3: Debug/Analyze

A junior developer says: "I installed numpy but Python says ModuleNotFoundError: No module named'numpy'". Give 3 possible causes and the diagnostic command for each.

Ans:-

1. Using the Wrong Version of Python

Most computers have multiple versions of Python installed. He might have installed NumPy for Python 3.10, but is trying to run his script using Python 3.12. Since each version has its own folder for "extras," the new version can't see what he has installed for the old one.

- Diagnostic Command: python --version (Compare this to the version shown when run pip --version)

2. Virtual Environment Not Activated

Developers often use "Virtual Environments" to keep projects separate. If he has installed NumPy while a virtual environment was active, but he is now trying to run the script in a global terminal (or vice versa), Python won't be able to find the module.

- Diagnostic Command: pip list (This shows every library the current environment can see.)

3. Pip and Python Are Not Linked

Sometimes the command pip points to a completely different location than the command python. This happens often on macOS and Linux. pip is probably sending it to a hidden system folder that his script doesn't check.

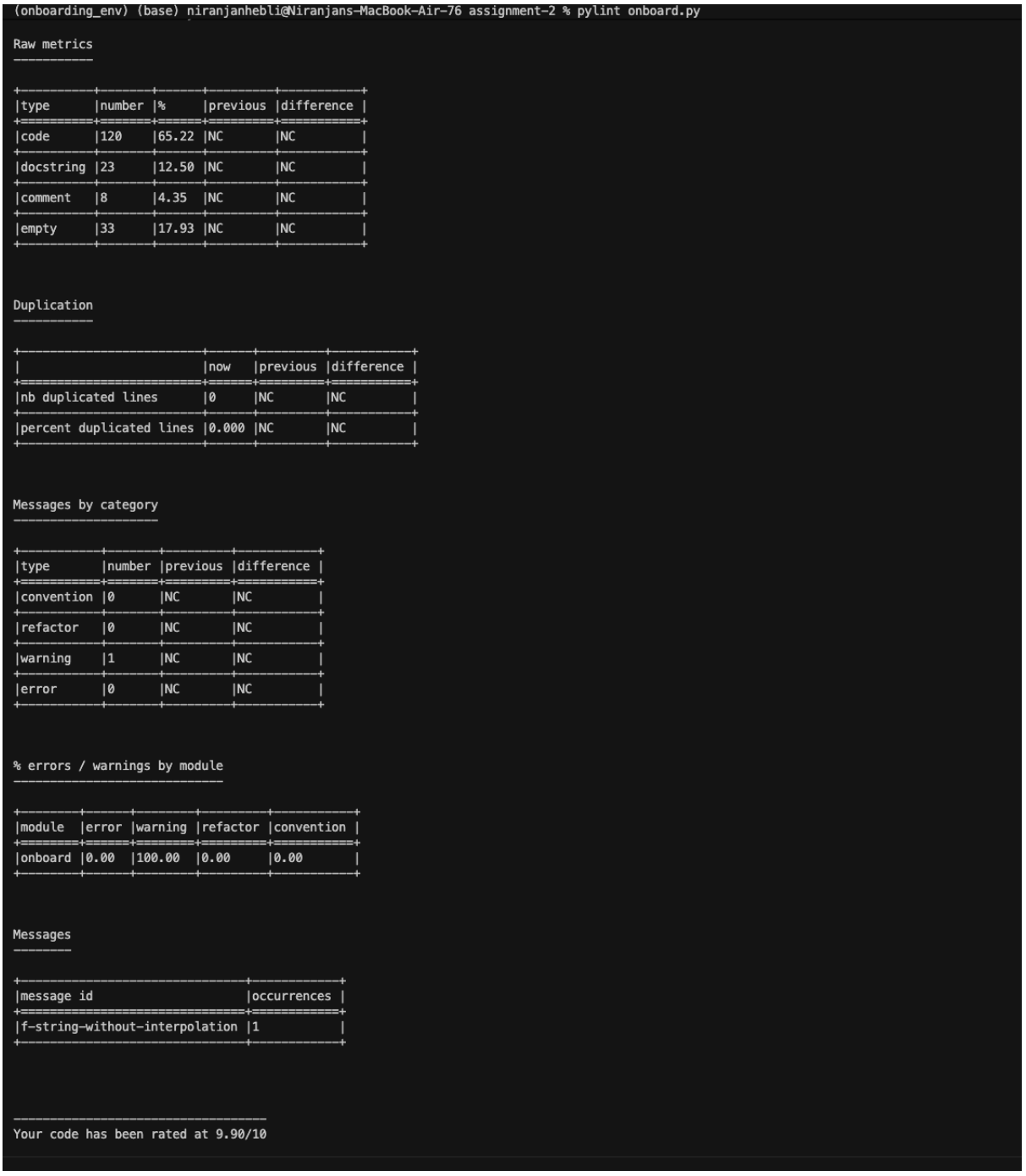
- Diagnostic Command: which python and which pip (On Windows, use where python and where pip). If the file paths lead to different folders, they are not linked.

Part D: AI-Augmented Task

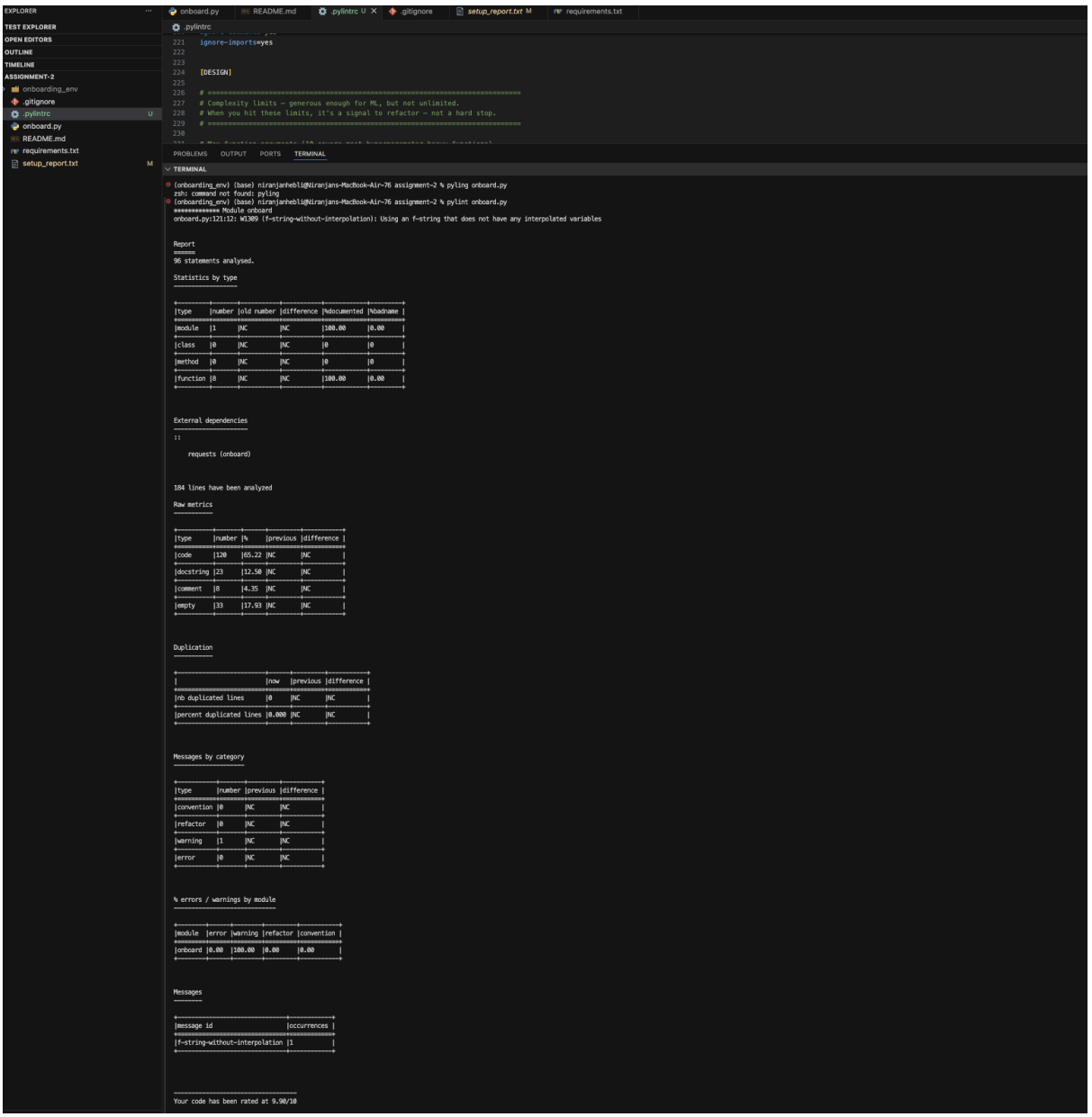
Prompt Used:-

Act as an expert Python developer. Generate a .pylintrc configuration file specifically for a beginner-level AI/Machine Learning project.
Please follow these requirements:
* Relaxed Style: Increase the max-line-length to 120 characters to accommodate long data processing strings and deep learning layers.
* Ignore Specific Warnings: Disable C0114, C0115, and C0116 (missing docstrings) so the beginner isn't forced to document every single function immediately.
* Naming Conventions: Adjust naming regex to be more lenient with variable names (e.g., allow short names like i, j, X, y, which are standard in ML).
* Similarity Threshold: Increase the min-similarity-lines to 10 to avoid flagging repetitive (but necessary) boilerplate code often found in model training loops.
* Plugin Support: Include pylint.extensions.docparams and ensure it plays well with common ML libraries like NumPy, Pandas, and PyTorch/TensorFlow. stay within the 100-line constraint while maintaining essential configuration guidance.

Github Link:-https://github.com/NiranjanHebli/DAY7_AM_Assignment/blob/main/.pylintrc



```
(onboarding_env) (base) niranjanhebli@Niranjans-MacBook-Air-76 assignment-2 % pylint onboard.py

Raw metrics
-----------

+----------+-------+------+---------+------------+
|type      |number |%     |previous |difference  |
+==========+=======+======+=========+============+
|code      |120    |65.22 |NC       |NC          |
+----------+-------+------+---------+------------+
|docstring |23     |12.50 |NC       |NC          |
+----------+-------+------+---------+------------+
|comment   |8      |4.35  |NC       |NC          |
+----------+-------+------+---------+------------+
|empty     |33     |17.93 |NC       |NC          |
+----------+-------+------+---------+------------+


Duplication
-----------

+-------------------------+------+---------+------------+
|                         |now   |previous |difference  |
+=========================+======+=========+============+
|nb duplicated lines      |0     |NC       |NC          |
+-------------------------+------+---------+------------+
|percent duplicated lines |0.000 |NC       |NC          |
+-------------------------+------+---------+------------+


Messages by category
--------------------

+-----------+-------+---------+------------+
|type       |number |previous |difference  |
+===========+=======+=========+============+
|convention |0      |NC       |NC          |
+-----------+-------+---------+------------+
|refactor   |0      |NC       |NC          |
+-----------+-------+---------+------------+
|warning    |1      |NC       |NC          |
+-----------+-------+---------+------------+
|error      |0      |NC       |NC          |
+-----------+-------+---------+------------+


% errors / warnings by module
-----------------------------

+--------+------+--------+---------+-----------+
|module  |error |warning |refactor |convention |
+========+======+========+=========+===========+
|onboard |0.00  |100.00  |0.00     |0.00       |
+--------+------+--------+---------+-----------+


Messages
--------

+-------------------------------+------------+
|message id                     |occurrences |
+===============================+============+
|f-string-without-interpolation |1           |
+-------------------------------+------------+



-----------------------------------
Your code has been rated at 9.90/10
```



What did the AI get right? What would you change? Did it include any rules that are too strict or too lenient for beginners?

What the AI got right:

- It correctly explained the purpose of a `.pylintrc` file and how it is used to configure Pylint.
- Showed the proper command to generate a starter configuration (`pylint --generate-rcfile > .pylintrc`).
- Highlighted useful customization options such as disabling warnings, setting maximum line length, and controlling naming conventions.
- Emphasized best practices like keeping `.pylintrc` under version control so teams share consistent rules.

What I would change:

- The AI's explanation was thorough, but for an assignment answer it could be **more concise** and focused on the essentials (purpose, creation, usage).
- Instead of listing too many advanced options (like regex for variable names), a simpler example would be clearer for beginners.
- I would include a **sample beginner-friendly** `.pylintrc` **snippet** with only the most common rules (line length, disabling docstring warnings, etc.) to make it more practical.

Rules too strict or too lenient for beginners:

- **Too strict:**
  - Enforcing complex naming regex rules may overwhelm beginners.
  - Requiring docstrings for every function/class is good practice but can be discouraging for those just learning syntax.

- **Too lenient:**
  - Allowing very long lines (default 100+) might make code harder to read; beginners benefit from learning shorter line lengths early.
  - Disabling too many warnings (like `invalid-name`) could prevent beginners from learning proper naming conventions.